

Time Complexity: $O(2^n)$

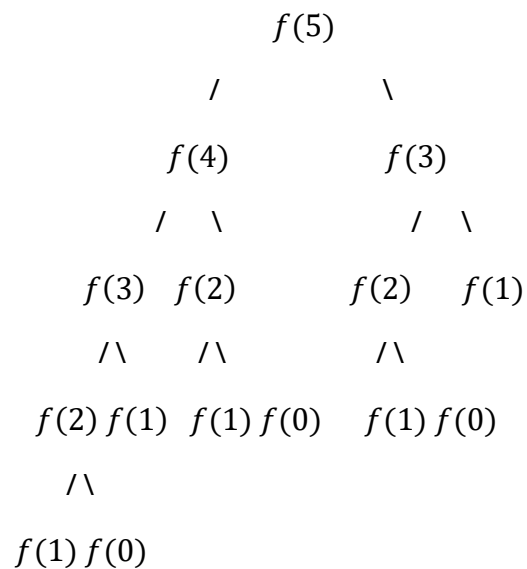
Justification:

The function calculates its result by calling itself two times in each step:

`hello_recursive(n-1)` and `hello_recursive(n-2)`. This process forms a binary recursion tree.

- **Base Case:** When $n \leq 1$, the function takes constant time $O(1)$.
- **Recursive Case:** For $n > 1$, the recurrence relation can be defined as $T(n) = T(n-1) + T(n-2) + c$ that can be simplified to $T(n) \leq 2 * T(n-1)$ by unrolling the relation it will result in that $T(n) = O(2^n)$.

OR



From the diagram we can conclude that since it's a binary tree it will almost has

Time Complexity: $O(2^n)$

Time Complexity: $O((\log(n))^2)$

Justification:

1. Outer Loop (while $i > 1$):

- The variable i is initialized to n and divided by two at the end of each iteration.
- The loop condition requires $i > 1$. Since i is an integer, the last valid value of i that enters the loop is 2 (assuming n is a power of two).
- Therefore, the sequence of values that i takes during the loop is: $n, \frac{n}{2}, \frac{n}{4}, \dots, 2$.
- This means the outer loop executes exactly $\log_2(n)$ times.
- Time Complexity: $O(\log(n))$

2. Inner Loop (while $j < i$):

- For any given value of i from the outer loop, the inner loop initializes $j = 1$ and doubles it ($j * 2$) until j reaches i .
- The number of operations performed by this inner loop is exactly $\log_2(i)$.

3. Total Number of Operations:

Since i changes in each iteration of the outer loop, the inner loop executes a different number of times per iteration. We must sum up these executions:

- **Iteration 1:** $i = n \Rightarrow$ Inner loop runs $\log_2(n)$ times.
- **Iteration 2:** $i = \frac{n}{2} \Rightarrow$ Inner loop runs $\log_2(\frac{n}{2}) = \log_2(n) - 1$ times.
- **Iteration 3:** $i = \frac{n}{4} \Rightarrow$ Inner loop runs $\log_2(\frac{n}{4}) = \log_2(n) - 2$ times.
- ...
- **Final Iteration:** $i = 2 \Rightarrow$ Inner loop runs $\log_2(2) = 1$ time.

The total number of all these operations is the sum of all these executions.

$$\text{Total} = 1 + 2 + 3 + 4 + \dots + \log_2(n)$$

And it can be simplified by $\frac{x(x+1)}{2}$ to be

$$\frac{\log_2(n) * \log_2(n + 1)}{2} \Rightarrow \frac{1}{2} * (\log_2(n))^2 + \frac{1}{2} * (\log_2(n))$$

So, Full-time complexity is: **$O((\log n)^2)$**